

PROJECT REPORT

B.Tech in Computer Science and Engineering

DIVISION OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Sanal Sivakumar

Project Details

Multi GNSS Dynamic Receiver Selection System
Adaptive reliability weights, live integration and simulation based evaluation

Guide Details

Project Guide

SEPTEMBER 2026

DIVISION OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CERTIFICATE

Certified that this is a Bonafide record of the project work done by **Sanal Sivakumar** (_____) of VIII Semester, Computer Science and Engineering in the year 2026 in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering of Cochin University of Science and Technology with distinction.

Project Guide

Head of Division

Acknowledgment

I would like to convey my heartfelt gratitude to all those who supported and guided me throughout this project. I am especially thankful to my project guide for constant support, valuable feedback and encouragement at every stage of this work. His guidance helped shape a vague idea into a complete, well-tested system.

I am profoundly grateful to the Head of Division for the academic environment and for allowing the use of the facilities needed to build and test the hardware and software components of this project. I also thank the faculty members whose courses in embedded systems, machine learning and probability provided the foundation for the techniques used here.

I extend sincere thanks to my classmates and friends for reviewing the design, pointing out ambiguities and helping me debug the trickier parts of the pipeline. Finally, I thank my parents for their un- wavering support, patience and for everything they have done for me.

Abstract

Low-cost GNSS receivers can disagree because satellite visibility, geometry, signal quality, multipath and stale observations vary over time. The project is built around a machine-learned weighting of the reliability indicators: the decision rule $W = a*T + b*S + c*SNR + d*DOP$ scores each receiver with four normalized indicators, and a compact neural network learns the coefficients $[a, b, c, d]$ from the current GNSS conditions, so the same self-explanatory formula is used everywhere while its weights adapt.

The hardware pipeline uses two GNSS data streams acquired by an ESP32 with a per-second C/N0 tracker, a Raspberry Pi decision engine that applies the learned weights with a switching deadband (hysteresis) and a fixed-weight fallback, and a Flask web view. The offline experiment conditions the weight network on 16 features derived from both receivers (raw reliability indicators, comparative deltas and short-term temporal statistics).

On the 20 held-out simulated test sessions, the learned weights (Model C) reached a mean selected error of 3.435 m, improving to 3.394 m with a 0.05 deadband and about 43 percent fewer source switches, compared with 3.509 m for the predefined fixed coefficients. These figures are results of the controlled simulation, not field or RTK accuracy claims.

Table of Contents

Acknowledgment	iii
Abstract	iv
List of Figures	vi
1. Introduction	1
1.1 Objectives	1
1.2 Scope	1
2. GNSS Background	2
2.1 How a GNSS position is formed	2
2.2 Why two receivers can disagree	2
2.3 Meaning of the main indicators	2
3. System Architecture	3
3.1 ESP32 message format	3
3.2 Processing components	4
4. The Weighted Reliability Score and Its Limitations	5
4.1 Why a numerical score is not a reliability percentage	5
4.2 Specific limitations of the predefined fixed set	5
5. Adaptive Weight Learning	6
5.1 Conditioning features	6
5.2 Network and parameter count	6
5.3 Why learn weights instead of predicting error directly	7
5.4 Deployment behaviour	7
6. Training and Evaluation Design	8
6.1 Training objective	8
6.2 Controlled scenario generation	8
6.3 Data split and leakage control	8
6.4 Optimisation and baselines	8
6.5 Metrics	8
7. Results	10
8. Results by Scenario and Learned-Weight Analysis	13
8.1 Feature ablation	14
8.2 Hysteresis sweep	14
9. Implementation Status and Remaining Gaps	15
9.1 Recommended sequence to a field claim	15
10. Conclusion	16
References	16
Appendix A Key Reproducibility Facts	17
Appendix B Selection Pseudocode	17

List of Figures

Fig 3.1 The live-integrated pipeline: the exported model drives the Raspberry Pi decision engine.	3
Fig 5.1 Learned-weight architecture: conditioning features produce the coefficients used by the $W = a*T + b*S + c*SNR + d*DOP$ rule.	6
Fig 7.1 Held-out comparison. Lower error is better.	10
Fig 7.2 Confusion matrix of the deployed selector (Model C + hysteresis 0.05) against the oracle on the 5,986 available test epochs.	11
Fig 8.1 Mean selected error by generated condition. Lower error is better.	13

1. Introduction

Global Navigation Satellite Systems estimate a receiver position from radio signals transmitted by satellites. A low-cost GNSS module normally reports latitude, longitude, time, satellite count and quality indicators. In open sky, two receivers may appear similar. In partly blocked or reflective surroundings, their reported positions can differ even when both still have a valid fix. A selection system is useful when it continuously decides which receiver output is more likely to be accurate at the current instant.

The project's decision rule is a weighted reliability score: each receiver gets a score $W = a \cdot T + b \cdot S + c \cdot \text{SNR} + d \cdot \text{DOP}$ built from timing accuracy, satellite count, signal-to-noise ratio and HDOP, and the receiver with the larger W is selected. The coefficients $[a, b, c, d]$ are not hand-tuned; they are learned by a compact neural network $f(X)$ that maps the current GNSS conditions to the weights. A predefined fixed set (0.25, 0.25, 0.30, 0.20) was used only during early bring-up so the acquisition and decision pipeline could be verified before the ML component was wired in; in the deployed system it survives solely as an emergency fallback and as a comparison baseline.

1.1 Objectives

- Acquire and format readings, including a 1 s average C/N0, from two GNSS receivers through an ESP32 UART bridge.
- Provide a Raspberry Pi decision engine and Flask path that applies the learned adaptive weights, with hysteresis and a fixed-weight fallback, and exposes a selected location.
- Learn the coefficients of the reliability equation with a compact neural network so the decision rule stays transparent while its weights become context-dependent.
- Evaluate the learned weights against the predefined fixed set and a globally optimized fixed set on held-out sessions, reporting mean, median, tail error, flip accuracy and switching behaviour.

1.2 Scope

The repository contains a hardware-oriented prototype whose live path is implemented and a separate offline model experiment that produced the exported weights. The ESP32 firmware, Raspberry Pi engine, web dashboard and the ML training/evaluation code are all in the repository. The evaluation reported here is a controlled simulation study; the model and engine are integrated together, but they have not yet been validated against synchronized real receiver logs and a trusted reference trajectory.

2. GNSS Background

2.1 How a GNSS position is formed

Each satellite broadcasts a time-stamped signal and orbital information. The receiver estimates the travel time of the signal and converts it to a pseudorange. With signals from at least four satellites, the receiver solves for three position coordinates and its clock bias. This is often called trilateration in introductory explanations, although the actual receiver solution uses pseudoranges, timing corrections and a least-squares or filter-based estimator.

2.2 Why two receivers can disagree

- **Visibility:** buildings, trees and the human body can block satellites differently for each antenna.
- **Geometry:** satellites clustered in one part of the sky produce a weaker geometric configuration than satellites spread widely across the sky.
- **Multipath:** a reflected signal travels a longer path and can bias the time estimate.
- **Receiver and antenna differences:** chipsets, antenna placement, update timing and firmware can change tracking quality.
- **Staleness:** a position can be valid but old; a moving platform needs fresh readings.

2.3 Meaning of the main indicators

Indicator	Meaning	Use in this project	Important caution
Satellite count	Number of satellites contributing to or visible for a solution.	Term S and a conditioning feature.	More satellites often helps, but quality and geometry still matter.
HDOP	Horizontal Dilution of Precision. Lower is generally better geometry.	Term DOP and a conditioning feature.	HDOP is a geometry indicator; it is not the actual position error.
C/N0	Carrier-to-noise density ratio in dB-Hz, a signal-quality measure (1 s average).	Term SNR and a conditioning feature.	Per-satellite aggregation is not implemented; one average is used.
Time accuracy	Seconds between the reported time and the live UTC clock.	Term T and an availability check.	A fresh timestamp does not guarantee low position error.
Position step	Distance from the prior reported position.	Conditioning feature and sanity indicator.	Large step can be motion, noise or a jump; context matters.

Table 1. GNSS indicators used in the reliability score and the adaptive weight network.

3. System Architecture

The data path begins with two GNSS modules. The ESP32 reads each NMEA stream through a separate hardware UART, parses it with TinyGPSPlus, tracks the per-satellite C/N0 from GSV sentences and emits one combined line through USB serial at about 1 Hz. The Raspberry Pi decision engine parses the line, evaluates the reliability terms, obtains adaptive weights [a,b,c,d] from the exported model (with a fixed-weight fallback), applies a hysteresis deadband and logs every decision to a JSONL file. A Flask endpoint serves the selected location to the browser dashboard.

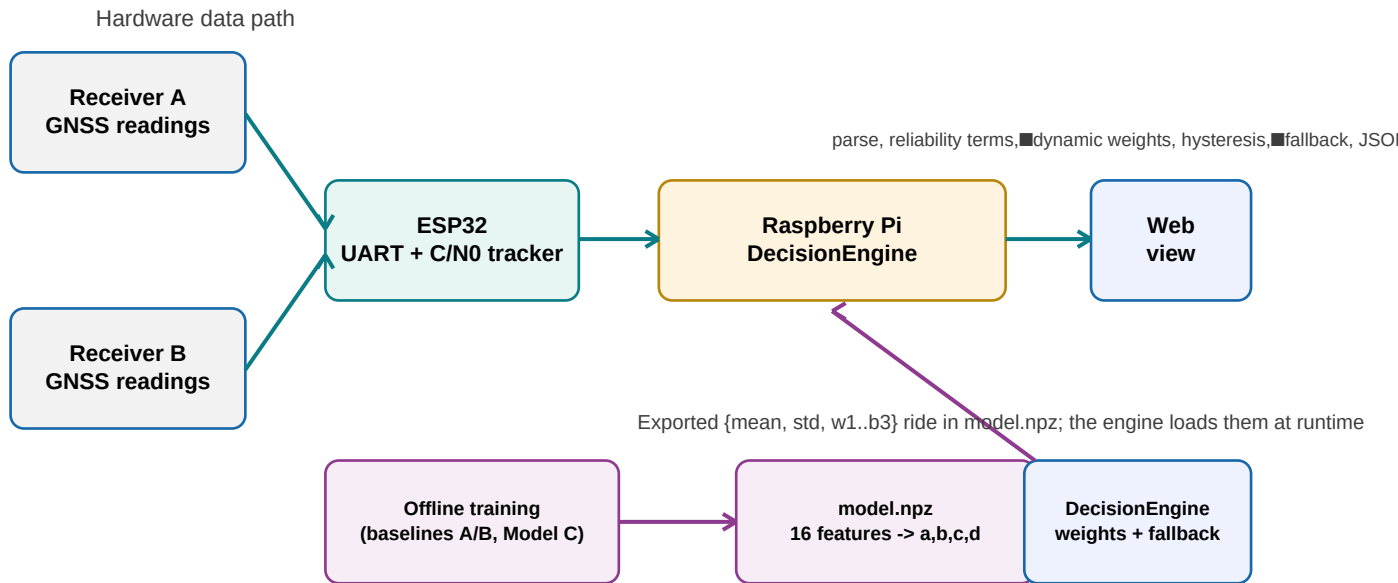


Fig 3.1 The live-integrated pipeline: the exported model drives the Raspberry Pi decision engine.

3.1 ESP32 message format

The firmware emits a compact, comma-separated representation for each receiver; the two records are separated with a vertical bar. Each record is GPS,time,lat,lon,sat,hdop,snr or GNSS,time,lat,lon,sat,hdop,snr, where snr is the 1 s average C/N0. Latitude and longitude become the string NO_FIX when location data is invalid; satellite count defaults to zero; HDOP defaults to 99.99.

GPS,12:10:05,10.051602,76.331617,18,0.82,46|GNSS,12:10:05,10.051610,76.331601,15,1.24,44

3.2 Processing components

Component	Technology	Responsibility	Current status
GNSS modules	NEO-M8N and Quectel L89	Produce receiver-specific GNSS observations.	Hardware path described in firmware; no field validation documented.
ESP32	Arduino C++ and TinyGPSPlus	Read two UART streams, track C/N0, publish a combined serial line.	Implements time, location, satellite count, HDOP and SNR output.
Decision engine	gnss_engine.py (NumPy inference)	Parse frames, compute terms, apply adaptive weights with hysteresis and fallback, log decisions.	Implemented and unit-tested; runs with a built-in simulated frame source.
Web view	Python, Flask and Leaflet	Serve the selected location and dashboard.	Implemented; /location returns live decision JSON.
Offline experiment	Python and NumPy	Generate controlled sessions, train Model C, grid-search Model B, export weights and metrics.	Implemented with a fixed seed; deterministic and CPU-only.

Table 2. Components and evidence-based implementation status.

4. The Weighted Reliability Score and Its Limitations

The decision rule of the project is the weighted reliability equation. Each receiver is scored with four components normalized to 0 to 1:

```
T = clip(1 - time_error / 5, 0, 1) S = clip(satellites / 20, 0, 1)
SNR = clip(cnr / 50, 0, 1) DOP = clip(1 / HDOP, 0, 1) W = a*T + b*S
+ c*SNR + d*DOP best_receiver = argmax(W)
```

4.1 Why a numerical score is not a reliability percentage

A value such as 0.8 is only the result of a formula unless it has been calibrated against repeated observed outcomes. It does not mean an 80 percent probability of being correct or an 80 percent chance of being within a distance. The project therefore reports the selection in measurable terms: mean, median and tail position error of the chosen receiver against the generated reference trajectory. The weights themselves are compared as numbers learned from data, not as probabilities.

4.2 Specific limitations of the predefined fixed set

Issue	Effect	Role in this project
Predefined 25/25/30/20 weights	No experimental basis for the relative importance of features; weights cannot adapt when the environment changes.	Model A keeps these as the fallback and comparison baseline; the learned model supplies the weights used in practice.
Satellite count capped at 20	Counts above 20 receive the same satellite term.	The cap is retained in the normalized term; additional satellites still appear in the 16-dimension conditioning input.
HDOP capped at 1	All HDOP values at or below 1 receive the same geometry term.	The cap remains in the term; raw HDOP enters the conditioning vector.
Satellite count and HDOP overlap	A hand formula can double-count closely related quality evidence.	The learned network can weight the terms jointly and nonlinearly, although correlation remains a data issue.
Invalid inputs inconsistently handled	Negative or stale values can create meaningless scores.	Eligibility rules reject invalid or stale receivers before scoring.
Weights cannot react to context	The relative value of timing versus geometry is assumed constant.	The adaptive network changes [a,b,c,d] with current conditions while keeping the equation interpretable.

Table 3. Motivation for learning context-dependent weights rather than relying on a predefined set.

5. Adaptive Weight Learning

The heart of the system is an adaptive weight learner. A compact MLP maps a conditioning feature vector X to softmax coefficients $[a,b,c,d] = f(X)$, which the reliability equation then uses to score each receiver. Constraining the output with softmax keeps a,b,c,d non-negative and summing to one, so the weights stay interpretable as a normalized allocation of importance. The learned model is the primary source of the coefficients; the predefined set is used only when the network is unavailable.

Trainable parameters: 1,108 (408 + 600 + 100). Stored normalization values: 32 (mean and std for 16 inputs). Output coefficients are the adaptive weights used

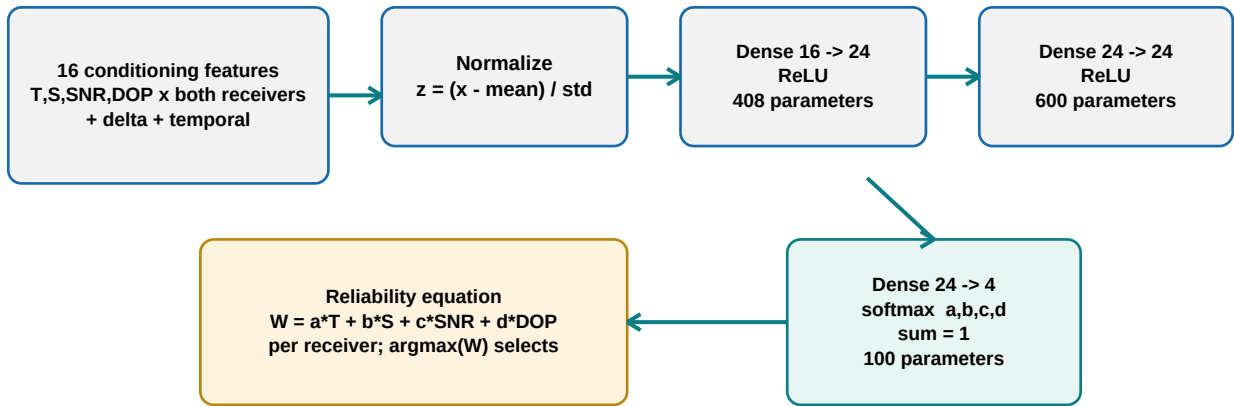


Fig 5.1 Learned-weight architecture: conditioning features produce the coefficients used by the $W = a*T + b*S + c*SNR + d*DOP$ rule.

5.1 Conditioning features

The network conditions on 16 features built from both receivers at each epoch: the eight raw reliability-relevant values (T-related timing error, satellites, C/N0 and HDOP for receiver 0 and receiver 1), four comparative deltas between the two receivers, and four temporal statistics (per-receiver satellite-count rate and a causal 5-sample C/N0 moving average). All features are observable at decision time. The model respects the project's raw channel order: satellites, HDOP, C/N0, age, step, time error and receiver id.

5.2 Network and parameter count

Layer	Shape	Activation	Parameters
Input	16	-	0
Dense 1	16 to 24	ReLU	$16 \times 24 + 24 = 408$
Dense 2	24 to 24	ReLU	$24 \times 24 + 24 = 600$
Output	24 to 4	softmax	$24 \times 4 + 4 = 100$
Total trainable	-	-	1,108
Stored normalization	mean and std for 16 features	-	32 non-trainable values

Table 4. Model C architecture and parameter accounting.

5.3 Why learn weights instead of predicting error directly

Predicting each receiver's error and choosing the smaller one is a valid alternative, but it changes the system's behaviour into a per-receiver regression on synthetic labels and requires trusting each network's error scale. Learning weights keeps the decision rule transparent and localized: the examiner can see that the formula is fixed and the network's whole role is to set its four coefficients.

5.4 Deployment behaviour

At inference the engine computes W for both receivers, restricts selection to receivers that pass the availability checks (valid fix, at least four satellites, HDOP inside range, data younger than 2 s), and applies a deadband: if the absolute difference $|W_0 - W_1|$ is below the hysteresis threshold, the previously selected receiver is retained to avoid rapid oscillation. If the learned model fails to load or returns abnormal output, the engine falls back to the predefined fixed coefficients (0.25, 0.25, 0.30, 0.20).

6. Training and Evaluation Design

6.1 Training objective

The network is trained with a soft, differentiable selection objective. Define the soft probability $P1 = \text{sigmoid}(\tau * (W1 - W2))$; the loss is the expected positioning error $E[P1 * E1 + P2 * E2]$ using the receivers' actual generated errors. As training proceeds the temperature τ anneals from 2.0 toward 0.5, making the soft selection progressively sharper toward the deterministic argmax used at deployment.

6.2 Controlled scenario generation

The generator creates 120 independent sessions of 300 seconds each and two receiver readings per second, giving 72,000 receiver readings. Scenarios include open sky, partial obstruction, urban multipath, dropout and mixed conditions. Each session includes a reference trajectory, reported observations, feature values, stale intervals, invalid fixes, shared drift, receiver-specific bias and occasional spikes. These are controlled assumptions used to study the decision logic, not measurements from a deployed GNSS system.

6.3 Data split and leakage control

Whole sessions are separated into 80 training, 20 validation and 20 test sessions, stratified per scenario, before any fitting. The feature mean and standard deviation are calculated from eligible training readings only, and the test split is untouched until the final evaluation.

Partition	Sessions	Purpose	Leakage rule
Training	80	Fit network parameters.	No session appears in validation or test.
Validation	20	Select the best epoch; grid-search Model B; stop early.	Whole held-out sessions.
Test	20	Report final independent results.	Never used to choose weights or epochs.

Table 5. Session-level partitioning used by the experiment.

6.4 Optimisation and baselines

Model C is trained with Adam (learning rate 0.001, batch 512, early stopping after 12 epochs without improvement, maximum 60 epochs). The best validation checkpoint was found at epoch 6. Two fixed-weight baselines accompany it: Model A reproduces the predefined fallback coefficients (0.25, 0.25, 0.30, 0.20); Model B is a fixed set found by an exhaustive grid search over the simplex at step 0.05 (1,771 candidates) on validation epochs, which selected α 0.10, β 0.20, γ 0.00 and δ 0.70.

6.5 Metrics

- **Mean, median, RMSE and 95th-percentile selected error:** distribution of the horizontal error of the receiver selected at each available epoch.

- **Flip accuracy vs oracle:** fraction of available epochs at which the selector agrees with the receiver that actually had the lower error (the oracle cannot be known live; it is an upper bound on decision quality).
- **Switches and mean duration:** number of source changes and average run length; with hysteresis the system trades a small amount of responsiveness for stability.
- **Availability:** number of epochs with at least one eligible receiver.

7. Results

The following results compare the predefined fixed set, the optimized fixed set and the learned adaptive model on the 20 held-out test sessions. All selection decisions use the same reliability equation; only the coefficients differ.

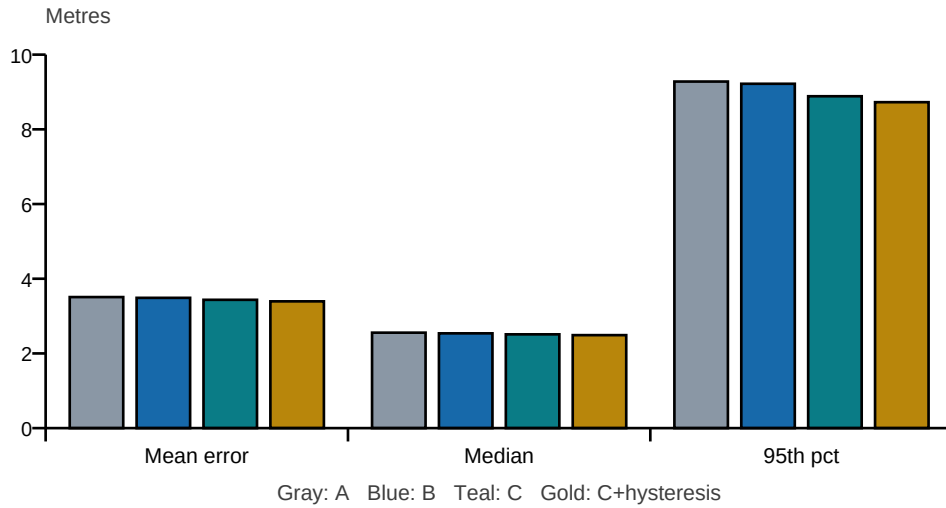


Fig 7.1 Held-out comparison. Lower error is better.

Selector	Accuracy vs oracle	Mean m	Median m	RMSE m	95th m	Switches
Model A - predefined fixed	61.3%	3.509	2.556	4.913	9.279	1,788
Model B - optimized fixed	61.6%	3.488	2.538	4.856	9.220	1,768
Model C - dynamic ML	62.6%	3.435	2.511	4.777	8.886	1,767
Model C + hysteresis (0.05)	64.0%	3.394	2.489	4.724	8.727	1,018

Table 6. Held-out results (5,986 available epochs). The oracle lower bound is not a deployable selector.

The row 'Accuracy vs oracle' counts, at each available epoch, whether the selector picked the receiver that actually had the lower error (64.0 percent for the deployed selector). The confusion matrix below breaks those decisions into full counts, with the lower-error receiver as the reference:

n = 5,986 available test epochs oracle agreement = 64.0%

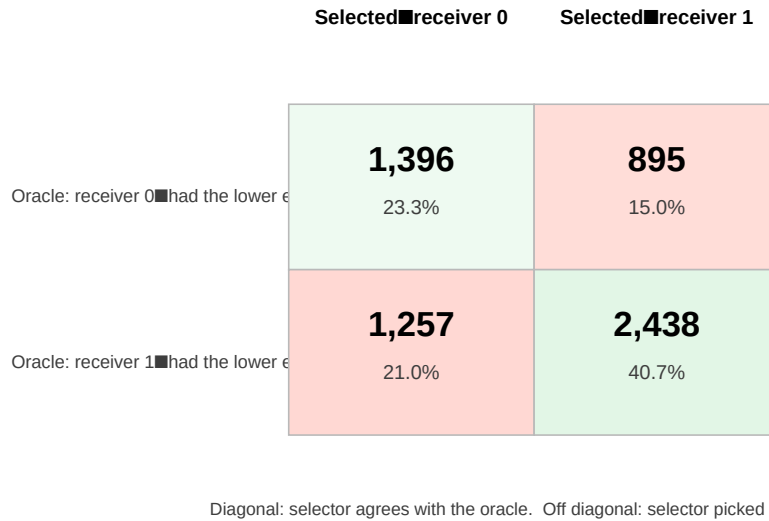


Fig 7.2 Confusion matrix of the deployed selector (Model C + hysteresis 0.05) against the oracle on the 5,986 available test epochs.

Selector	Oracle R0, sel R0	Oracle R0, sel R1	Oracle R1, sel R0	Oracle R1, sel R1	Oracle agreement
Model A - predefined fixed	1,455	836	1,482	2,213	61.3%
Model B - optimized fixed	1,466	825	1,475	2,220	61.6%
Model C - dynamic ML	1,363	928	1,310	2,385	62.6%
Model C + hysteresis (0.05)	1,396	895	1,257	2,438	64.0%

Table 7. Selector-vs-oracle confusion counts over the held-out test split. 'Oracle R0' means receiver 0 had the lower actual error at that epoch; 'sel R1' means the rule picked receiver 1. Diagonal columns are agreement with the oracle.

In absolute terms the deployed selector agrees with the oracle at 64.0 percent of epochs. It is conservative in a specific way: it agrees on more than half of the epochs where the oracle prefers receiver 1, while a fixed baseline struggles with the same set. The off-diagonal imbalance also shows why the reliability equation needs a hysteresis deadband: many of the residual disagreements occur when the two scores are close, which is exactly the regime hysteresis suppresses.

Model C reduced mean selected error by 2.1 percent relative to the predefined fixed Model A and by 1.5 percent relative to the optimized fixed Model B. With the 0.05 hysteresis deadband, mean error fell to 3.394 m, 95th-percentile error to 8.727 m, flip accuracy rose from 61.3 percent (A) to 64.0 percent, and source switches dropped by 43.1 percent compared with Model A.

The honest reading is that the accuracy gain of dynamic weighting over even the optimized fixed set is small. Most of the practical improvement comes from the hysteresis deadband, which stabilizes the output track rather than dramatically changing accuracy. These figures only support the claim

that the adaptive weights learned the patterns built into this controlled study.

8. Results by Scenario and Learned-Weight Analysis

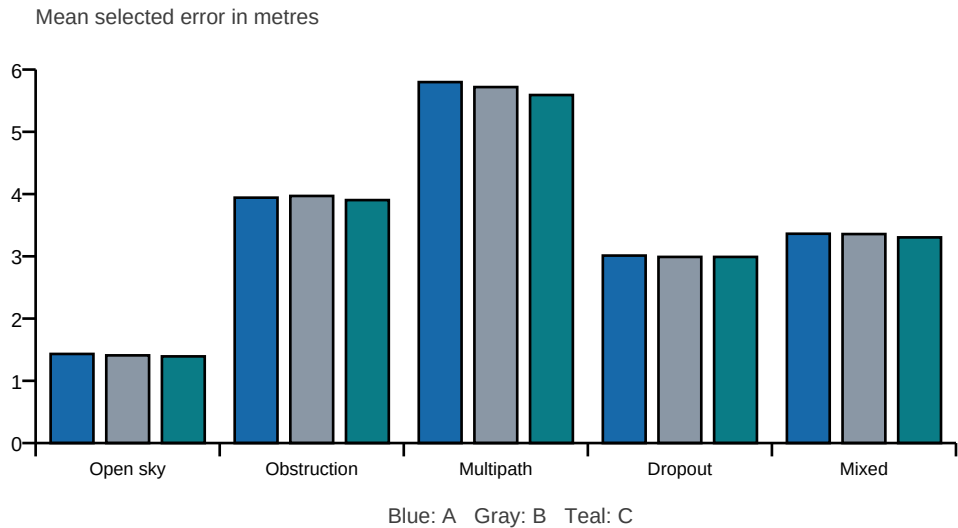


Fig 8.1 Mean selected error by generated condition. Lower error is better.

Scenario	Model A mean m	Model B mean m	Model C mean m
Open sky	1.432	1.409	1.392
Obstruction	3.942	3.970	3.903
Multipath	5.799	5.719	5.591
Dropout	3.012	2.990	2.990
Mixed	3.363	3.357	3.304

Table 8. Scenario comparison on the held-out test split.

The largest margin occurs in the urban-multipath condition, where Model C can combine several degraded indicators. The adaptive weights are also interpretable, which is a core claim of the project:

Condition (test, both receivers available)	alpha (timing)	beta (sats)	gamma (SNR)	delta (DOP)
Open sky (4344 epochs)	0.081	0.168	0.013	0.737
Urban multipath (423 epochs)	0.419	0.571	0.004	0.007
Good geometry (1501 epochs)	0.003	0.004	0.000	0.992
Poor geometry (960 epochs)	0.393	0.570	0.007	0.030

Table 9. Learned weights by proxy condition, aggregated over test epochs (see metrics.json learned_weights for the full set).

Overall, the model allocates about 58 percent of its weight to DOP and 15 percent to timing, but in open-sky epochs the DOP weight approaches 1.00 while in urban multipath the timing and satellite weights dominate (alpha + beta ~0.96). The gamma (SNR) term carries little extra discriminative power in this synthetic data. These weight shifts are correlational observations about the generated study, not causal claims.

8.1 Feature ablation

Conditioning input	Mean error m	Flip accuracy	Switches
T only	3.504	61.4%	1,839
S only	3.457	62.2%	1,758
SNR only	3.504	61.6%	1,868
DOP only	3.518	61.4%	1,901
T + S	3.505	61.4%	1,868
T + S + SNR	3.481	61.5%	1,719
all four	3.468	62.2%	1,842
all four + comparative	3.450	62.5%	1,757
all four + temporal (full)	3.440	62.8%	1,750

Table 10. Ablation of conditioning feature groups (30-epoch budget, test split). Adding comparative and temporal features progressively improves selection.

8.2 Hysteresis sweep

Threshold	Accuracy vs oracle	Mean m	Switches	Mean run s
0.00	62.6%	3.435	1,767	3.33
0.02	63.6%	3.413	1,314	4.46
0.05	64.0%	3.394	1,018	5.72
0.10	63.8%	3.407	716	8.05

Table 11. Hysteresis sweep for Model C. Threshold 0.05 was chosen for balance: better accuracy and error with about 42 percent fewer switches than no deadband.

9. Implementation Status and Remaining Gaps

A defensible project report states both what is implemented and what remains before a field-ready system. The live path is now implemented end to end: firmware with C/N0, a unit-tested decision engine with adaptive weights, hysteresis and logging, and a simulated frame source for hardware-free verification. The offline experiment is complete and deterministic. The remaining work is empirical validation.

Area	Status in current source	Remaining work
ESP32 firmware	Two UART parsers, HDOP and 1 s average C/N0 output; new 7-field frame.	Field check on real receivers; confirm timing synchronization and C/N0 calibration.
Decision engine	gnss_engine.py: parsing, reliability terms, adaptive weights, hysteresis, fixed-weight fallback, JSONL logging; 8 unit tests pass.	Tune the hysteresis threshold and fallback behaviour from real data.
Web service	server.py runs the engine with --simulate or serial; /location serves decision JSON.	Serial reconnection and health reporting need field testing.
ML experiment	Deterministic NumPy training (Model A/B/C), grid search, ablation, hysteresis sweep; 9 tests pass.	Retrain on synchronized real sessions separated by route or time.
Validation	Simulated reference trajectories and generated errors only.	Collect real reference positions (RTK or surveyed), remove faulty reference intervals, account for antenna offsets.

Table 12. Implementation status and evidence boundaries.

9.1 Recommended sequence to a field claim

- 1 Collect synchronized raw messages, parsed features, selected source, weights, fix state and switches into the existing JSONL log.
- 2 Build a synchronized real reference trajectory (RTK or surveyed points) and remove faulty reference intervals.
- 3 Retrain and retest by whole real sessions, separated by route or time period.
- 4 Re-tune the hysteresis threshold and fallback behaviour from the real sessions.
- 5 Report mean, median, tail error, availability and switching by environment, not only overall numbers.

10. Conclusion

This project is a dynamic receiver-selection system built around a transparent decision rule - $W = a*T + b*S + c*SNR + d*DOP$ with argmax selection - whose weights are learned adaptively by a compact neural network. The offline experiment shows that the network learns interpretable, context-dependent weights: DOP dominates under good conditions, while timing and satellite terms take over under degraded ones. The gains over even an optimized fixed set are modest, and most of the practical stability comes from the hysteresis deadband.

The live pipeline integrates all pieces: the ESP32 emits C/N0, the Raspberry Pi engine applies learned weights with fallback and hysteresis and logs every decision, and the dashboard visualizes results. The next milestone is not more modelling: it is collecting synchronized real receiver and reference data so that the same pipeline can be retrained and honestly evaluated outside the controlled simulation.

References

- 1 Project source repository: ESP32 firmware, Raspberry Pi decision engine and Flask service, offline model code and exported metrics, inspected September 2026.
- 2 E. D. Kaplan and C. J. Hegarty, Understanding GPS GNSS Principles and Applications, 3rd ed., Artech House, 2017.
- 3 P. Misra and P. Enge, Global Positioning System Signals Measurements and Performance, 2nd ed., Ganga-Jamuna Press, 2011.
- 4 NumPy project documentation and the repository-pinned NumPy 2.4.6 runtime used for the experiment.
- 5 M. Hart, TinyGPSPlus Arduino library documentation, used for the ESP32 parsing interface.

Appendix A Key Reproducibility Facts

Item	Recorded value
Training backend	numpy (NumPy 2.4.6)
Random seed	20260913
Sessions and readings	120 sessions; 300 seconds each; 72,000 receiver readings
Split	80 train / 20 validation / 20 held-out test sessions (stratified per scenario)
Decision-relevant training epochs (both available)	22436
Best validation epoch	Epoch 6 of 60 max; patience 12
Temperature schedule	annealed 1.975 toward 1.55
Model B search	1771 candidates; validation mean error 3.7216 m
Hysteresis used	0.05
Training CSV	readable rows of the 80 training sessions
Testing CSV	readable rows of the 20 test sessions

Table A1. Information needed to reproduce the recorded offline experiment.

Appendix B Selection Pseudocode

```

for each epoch, for each receiver i:
    check eligibility (fix valid, sat >= 4, HDOP in range, age <= 2)
    T_i = clip(1 - time_err_i/5, 0, 1)
    S_i = clip(sat_i/20, 0, 1)
    SNR_i = clip(cn0_i/50, 0, 1)
    DOP_i = clip(1/hdop_i, 0, 1)

[a, b, c, d] = f(X) # learned, softmax, sum = 1
W_i = a*T_i + b*S_i + c*SNR_i + d*DOP_i
if |W0 - W1| < hysteresis: keep the previous receiver
selected = argmax(W) over available receivers (else none)
publish selected source, coordinates, weights and reason

```

The result is a source-selection decision. The model does not correct a coordinate and does not fuse two positions; it chooses one receiver's reported position.